

Bot Telegram Katalog Disk/CD Backup Transaksi

Bot ini adalah sistem katalog bersama untuk mencatat **TID**, beberapa foto disk/CD backup, dan keterangan. Bot menggunakan Python dengan `python-telegram-bot`, Supabase sebagai database, serta `file_id` Telegram sehingga foto tidak perlu diunggah ke penyimpanan eksternal. Pola `ApplicationBuilder` dan polling yang digunakan mengikuti API modern `python-telegram-bot` 1, sedangkan operasi insert/select Supabase mengikuti client Python resminya 2 3.

1. Fitur

Fitur	Perilaku
Melihat data	Pengguna mengirim TID berupa angka; bot mengirim TID, seluruh foto, dan keterangan.
Daftar TID baru	Pengguna ketik <code>Daftar</code> , mengirim TID, satu atau beberapa foto, lalu keterangan.
TID belum ada	Jika pengguna mengirim TID yang belum tercatat, bot menawarkan tombol untuk mendaftarkannya.
Edit gambar	Pengguna ketik <code>Edit</code> , memilih TID, lalu dapat mengganti semua gambar lama.
Tambah gambar	Pengguna ketik <code>Edit</code> , memilih TID, lalu dapat menambahkan gambar tanpa menghapus gambar lama.
Edit keterangan	Pengguna ketik <code>Edit</code> , memilih TID, lalu mengirim keterangan baru.
Pembatalan	Ketik <code>Batal</code> pada proses apa pun untuk kembali ke keadaan awal.
Kontrol admin	Admin yang sesuai <code>ADMIN_ID</code> dapat membuka <code>/admin</code> , mengatur maintenance, mengaktifkan/menonaktifkan pendaftaran dan edit, serta melihat statistik.

Bot ini memakai mode **polling**, sehingga Railway menjalankan satu proses Python yang terus menerima pesan dari Telegram. Untuk penggunaan banyak orang, database menjadi

sumber data bersama dan state percakapan disimpan per pengguna oleh `ConversationHandler` .

Catatan akses: Versi ini bersifat katalog bersama. Setiap pengguna yang dapat mengakses bot dan mengetahui TID dapat melihat data TID tersebut, dan setiap pengguna dapat mengeditnya. Akses admin dibatasi secara ketat berdasarkan Telegram user ID pada `ADMIN_ID` . Pengguna biasa tetap dapat melihat data ketika bot aktif, tetapi tidak dapat membuka menu admin. Jika diperlukan pembatasan berdasarkan grup, daftar user tambahan, atau audit log, tambahkan lapisan otorisasi sebelum dipakai pada lingkungan produksi.

2. Struktur berkas

Plain Text

```
telegram-backup-catalog/  
├─ bot.py  
├─ requirements.txt  
├─ Procfile  
├─ schema.sql  
└─ README.md
```

3. Prasyarat

Siapkan akun Telegram, akun Supabase, dan akun Railway. Untuk pengujian lokal, siapkan Python 3.10 atau lebih baru. Railway mendukung penentuan start command melalui konfigurasi deployment [4](#) .

4. Membuat bot di BotFather

1. Buka Telegram dan cari akun resmi **@BotFather**.
2. Kirim `/newbot` .
3. Masukkan nama tampilan bot, misalnya `Katalog Backup Transaksi` .
4. Masukkan username bot yang diakhiri `bot` , misalnya `katalog_backup_bot` .
5. BotFather akan memberikan token. Simpan token tersebut sebagai secret dan jangan memasukkannya ke GitHub atau mengirimkannya ke orang lain.
6. Cari Telegram user ID milik calon admin. Cara praktisnya adalah mengirim pesan ke bot terpercaya seperti `@userinfobot` atau memakai mekanisme internal organisasi. Nilai

yang diperoleh harus berupa angka, misalnya `123456789` , dan disimpan sebagai `ADMIN_ID` .

7. Untuk memudahkan pengguna, atur perintah dengan `/setcommands` , lalu masukkan:

Plain Text

```
start - Menampilkan panduan bot
help - Menampilkan panduan bot
daftar - Mendaftarkan TID baru
edit - Mengedit gambar atau keterangan
batal - Membatalkan proses berjalan
```

5. Menyiapkan Supabase

5.1 Membuat project

1. Buka supabase.com dan buat akun atau masuk.
2. Buat project baru.
3. Pilih organisasi, nama project, password database, dan region yang sesuai.
4. Tunggu sampai project selesai dibuat.

5.2 Membuat tabel

1. Buka menu **SQL Editor** pada project Supabase.
2. Buka file `schema.sql` dari paket ini.
3. Salin seluruh isinya ke SQL Editor.
4. Klik **Run**.
5. Periksa menu **Table Editor** dan pastikan tabel berikut tersedia:

Tabel	Isi
<code>tid_records</code>	Satu baris untuk setiap TID dan keterangan terbarunya.
<code>tid_photos</code>	Satu baris untuk setiap foto, termasuk <code>file_id</code> Telegram dan urutannya.
<code>bot_settings</code>	Satu baris settings global untuk maintenance, pendaftaran, dan fitur edit.

5.3 Mengambil URL dan key

1. Buka **Project Settings** → **API**.
2. Salin **Project URL** sebagai nilai `SUPABASE_URL`.
3. Untuk deployment server-side ini, gunakan key rahasia dengan hak server, biasanya bernama **service_role key**, sebagai `SUPABASE_KEY`. Key tersebut diperlukan karena SQL mengaktifkan Row Level Security tanpa policy publik.
4. **Jangan** memakai service role key pada frontend, jangan memasukkannya ke repository, dan jangan membagikannya melalui chat publik. Key hanya disimpan sebagai environment variable Railway.

5.4 Penjelasan desain database

Tabel `tid_records` menggunakan TID sebagai primary key. Tabel `tid_photos` menggunakan foreign key ke TID dan `on delete cascade`, sehingga foto terkait ikut terhapus apabila suatu saat record TID dihapus. Satu foto disimpan sebagai satu baris agar satu TID dapat memiliki jumlah foto lebih dari satu. Foto fisik tetap berada di server Telegram; database hanya menyimpan `file_id` yang dikembalikan Telegram.

6. Konfigurasi environment variable

Bot membutuhkan empat variable berikut:

Variable	Isi
<code>BOT_TOKEN</code>	Token yang diberikan BotFather.
<code>SUPABASE_URL</code>	Project URL dari Supabase.
<code>SUPABASE_KEY</code>	Service role key Supabase untuk proses server-side.
<code>ADMIN_ID</code>	Telegram user ID numerik milik admin utama.

Jangan menulis nilai sebenarnya ke `bot.py`. Untuk pengujian lokal, buat file `.env` hanya jika Anda menggunakan loader tambahan; versi kode ini membaca environment variable langsung. Contoh perintah Linux/macOS:

Bash

```
export BOT_TOKEN="123456:token-dari-botfather"
export SUPABASE_URL="https://project-id.supabase.co"
```

```
export SUPABASE_KEY="service-role-key-supabase"
export ADMIN_ID="123456789"
python bot.py
```

7. Menjalankan lokal

Dari direktori project, jalankan:

Bash

```
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python bot.py
```

Jika berhasil, log akan menampilkan bahwa bot berjalan dalam mode polling. Hentikan dengan `Ctrl+C`. Jangan menjalankan dua instance polling dengan token bot yang sama secara bersamaan karena Telegram hanya boleh dilayani oleh satu proses aktif untuk konfigurasi polling tersebut.

8. Deploy ke Railway

8.1 Pilihan menjalankan bot

Pilihan	Kelebihan	Kekurangan	Cocok untuk
Railway dengan Procfile	Berjalan sebagai service online, tidak bergantung pada komputer pribadi, dan mudah dihubungkan ke repository GitHub.	Perlu memantau batas paket dan kebijakan harga Railway yang berlaku saat deployment.	Penggunaan bersama dan operasional harian.
Menjalankan lokal	Paling sederhana untuk uji coba dan debugging.	Komputer harus tetap menyala dan koneksi internet harus aktif.	Pengujian awal atau penggunaan pribadi sementara.

Untuk bot yang dipakai banyak orang, gunakan Railway atau platform hosting always-on setara. Railway dapat membaca Procfile pada root project; file pada paket ini menjalankan `python bot.py` sebagai worker process 4. Selalu periksa halaman harga dan kebijakan terbaru Railway karena paket gratis dan batas pemakaian dapat berubah.

8.2 Deployment melalui GitHub

1. Buat repository GitHub baru, misalnya `telegram-backup-catalog` .
2. Upload `bot.py` , `requirements.txt` , `Procfile` , `schema.sql` , dan `README.md` .
3. Masuk ke railway.app dan buat project baru.
4. Pilih **Deploy from GitHub repo**, lalu pilih repository tersebut.
5. Buka service yang dibuat, masuk ke **Variables**, dan tambahkan:

Plain Text

```
BOT_TOKEN=token-dari-BotFather
SUPABASE_URL=https://project-id.supabase.co
SUPABASE_KEY=service-role-key-supabase
ADMIN_ID=123456789
```

1. Pastikan `Procfile` ada di root repository dan berisi:

Plain Text

```
worker: python bot.py
```

1. Jalankan deployment. Buka **Deployments** → **Logs** dan cari pesan bahwa bot sudah berjalan.
2. Uji dengan mengirim `/start` ke bot. Admin juga dapat mengirim `/admin` untuk memastikan menu kontrol muncul.

8.3 Deployment melalui upload project

Jika tidak memakai GitHub, buat project Railway lalu gunakan alur upload/source yang tersedia pada dashboard Railway. Pastikan seluruh file berada di root source dan environment variable sudah diisi sebelum deployment. Bila Railway meminta start command secara manual, gunakan:

Plain Text

```
python bot.py
```

9. Cara memakai bot

9.1 Melihat TID yang sudah ada

Kirim angka, misalnya:

Plain Text

123456

Bot akan mengirim ringkasan TID, jumlah foto, keterangan, lalu semua foto yang tersimpan. Jika foto lebih dari sepuluh, bot mengirimkannya dalam beberapa batch agar sesuai batas pengiriman media Telegram.

9.2 Mendaftarkan TID baru

Kirim:

Plain Text

Daftar

Kemudian ikuti urutan berikut:

1. Kirim TID berupa angka.
2. Kirim foto disk/CD satu per satu sebanyak yang diperlukan.
3. Ketik **Selesai** setelah semua foto terkirim.
4. Kirim keterangan, misalnya periode transaksi, lokasi penyimpanan, jenis media, atau informasi rekonsiliasi.
5. Bot menyimpan data dan menampilkan kembali seluruh data TID.

Jika Anda langsung mengirim TID yang belum ada, bot juga akan menampilkan tombol **Daftarkan TID ini**.

9.3 Mengedit data

Kirim:

Plain Text

Edit

Lalu:

1. Kirim TID yang ingin diubah.
2. Pilih **Ubah semua gambar**, **Tambah gambar**, atau **Ubah keterangan**.
3. Untuk gambar, kirim satu atau beberapa foto dan ketik **Selesai**.

4. Untuk keterangan, kirim teks keterangan baru.
5. Bot akan menampilkan data terbaru setelah perubahan disimpan.

9.4 Membatalkan proses

Ketik:

Plain Text

Batal

Perintah ini menghapus state percakapan sementara pada sesi pengguna dan tidak menghapus data yang sudah tersimpan di database.

10. Fitur admin

10.1 Membuka menu admin

Pastikan Telegram user ID admin sudah disimpan pada environment variable `ADMIN_ID`. Setelah deployment, admin mengirim:

Plain Text

/admin

Bot menampilkan menu inline dengan status tiga setting global. Hanya user dengan ID yang sama persis dengan `ADMIN_ID` yang dapat membuka menu atau menekan tombolnya. User lain menerima pesan akses ditolak.

10.2 Kontrol yang tersedia

Tombol	Dampak
Aktifkan maintenance	Semua user biasa menerima pesan Bot sedang dalam maintenance, silakan coba lagi nanti dan tidak dapat mencari, mendaftar, atau mengedit data. Admin tetap dapat membuka /admin.
Nonaktifkan maintenance	Bot kembali melayani user biasa.
Nonaktifkan pendaftaran	Perintah <code>Daftar</code> dan tombol pendaftaran TID baru ditolak; data yang sudah ada tetap dapat dilihat.

Aktifkan pendaftaran	Pendaftaran TID baru tersedia kembali.
Nonaktifkan edit	Perintah <code>Edit</code> dan proses edit ditolak; data yang sudah ada tetap dapat dilihat.
Aktifkan edit	Fitur edit tersedia kembali.
Lihat statistik	Menampilkan jumlah TID terdaftar dan jumlah foto total.
Refresh menu	Membaca ulang status terbaru dari Supabase.

10.3 Persistensi status

Status disimpan pada tabel `bot_settings` di Supabase, bukan hanya di memori proses. Karena itu, status maintenance, pendaftaran, dan edit tetap berlaku setelah bot restart atau deployment ulang. Jalankan ulang `schema.sql` agar tabel `bot_settings` dibuat dan memiliki satu baris konfigurasi awal.

11. Error handling dan perilaku aman

Kode melakukan validasi bahwa TID hanya berisi angka, menolak keterangan kosong, menangani kegagalan Supabase dengan pesan yang mudah dipahami, mencatat detail error pada log Railway, serta menggunakan fallback pengiriman satu-per-satu apabila pengiriman album foto gagal. Query database dijalankan di thread terpisah agar operasi client Supabase yang sinkron tidak menahan event loop bot.

Jika dua pengguna mencoba mendaftarkan TID yang sama pada waktu hampir bersamaan, primary key database mencegah duplikasi. Salah satu proses akan gagal dan pengguna menerima pesan untuk mencoba lagi.

12. Troubleshooting

Gejala	Pemeriksaan
Bot tidak merespons	Periksa <code>BOT_TOKEN</code> , status deployment Railway, dan log service. Pastikan hanya satu instance bot yang aktif.
<code>SUPABASE_URL</code> atau key error	Pastikan URL tidak memiliki spasi dan key berasal dari project yang sama.
<code>/admin</code> ditolak untuk admin	Pastikan <code>ADMIN_ID</code> berisi Telegram user ID numerik, bukan username atau nomor

	telepon, lalu redeploy/restart service.
Menu admin gagal dibuka	Pastikan tabel <code>bot_settings</code> sudah dibuat dari <code>schema.sql</code> dan service memakai service role key server-side.
User biasa masih dapat daftar/edit	Pastikan setting yang diinginkan sudah dinonaktifkan dari <code>/admin</code> , dan cek log apakah deployment terbaru sudah aktif.
Error permission Supabase	Gunakan service role key hanya di environment variable server, atau buat RLS policy yang sesuai jika memakai key anon.
TID tersimpan tetapi foto tidak tampil	Pastikan pesan foto benar-benar dikirim sebagai foto Telegram, bukan file dokumen, dan lihat log Railway.
Pendaftaran gagal setelah foto	Pastikan <code>schema.sql</code> sudah dijalankan seluruhnya dan tabel <code>tid_records</code> serta <code>tid_photos</code> tersedia.
Deployment berhenti	Periksa log build, versi Python yang dipilih Railway, dan pastikan <code>requirements.txt</code> serta <code>Procfile</code> berada di root repository.

13. Catatan produksi untuk data perbankan

Data backup transaksi perbankan bersifat sensitif. Gunakan kontrol akses Telegram yang sesuai, batasi siapa yang boleh berada di grup atau mengakses bot, jangan membagikan token bot maupun service role key, dan pertimbangkan kebijakan retensi serta audit organisasi. Versi awal ini tidak mengimplementasikan enkripsi tambahan, approval workflow, penghapusan terjadwal, atau pembatasan hak edit per user; fitur tersebut perlu ditambahkan jika diwajibkan oleh kebijakan keamanan internal.

Referensi

- [1] Dokumentasi python-telegram-bot
- [2] Supabase Python client introduction
- [3] Supabase Python insert reference
- [4] Railway: Set a Start Command